DSCI 554 Lab 3

Power Analysis, Early Stopping in A/B Testing, and Causality Through Observational Studies

Contents

Lab Overview **3**

Lab Mechanics **3**

Code Quality **3**

Writing **3**

A Note on Challenging Questions **3**

Setup **4**

Exercise 1: Understanding and Calculating Power **5**
- Q1.1. Formulating the Hypotheses . . . 5
- Q1.2. Computing the Standard Error . . . 5
- Q1.3. Plotting the Null Distribution of $\hat{\delta}$ . . . 6
- Q1.4. Plotting Critical Value on the Null Distribution . . . 7
- Q1.5. Plotting the Alternative Distribution of $\hat{\delta}$ . . . 8
- Q1.6. Calculating Power with 100 Experimental Units per Treatment . . . 9
- Q1.7. Increasing to $1,000$ Experimental Units per Treatment . . . 10
- Q1.8. Calculating Power with $1,000$ Experimental Units per Treatment . . . 11
- Q1.9. Final Comments on Increasing the Sample Size . . . 12

Exercise 2: What Sample Size to Use **13**
- Q2.1. First Manual Sample Size Computation per Treatment . . . 13
- Q2.2. Second Manual Sample Size Computation per Treatment . . . 13
- Q2.3. Sample Size Computation per Treatment using Package `pwr` . . . 15
- (Challenging) Q2.4. A More Comprehensive Power Analysis Simulation . . . 16

Exercise 3: Basics of Early Stopping in Hypothesis Testing **20**
- Q3.1. p-value Progress in Sequential Two-sample t-tests . . . 21
- Q3.2. No Early Stopping when the Null Hypothesis is True . . . 22
- Q3.3. Early Stopping when the Null Hypothesis is True . . . 24
- (Challenging) Q3.4. Extending the Early Stopping Simulation . . . 28
- (Challenging) Q3.5. Multiple Simulations with Early Stopping . . . 30
- (Challenging) Q3.6. Comment on Your Findings . . . 37

Exercise 4: Introduction to Observational Studies in Causal Inference **38**
- Q4.1. Confounders in an Observational Study . . . 38
- Q4.2. Causal Analysis in an Observational Study . . . 39

Submission **40**

Attribution **40**

Lab Overview

In this lab, you will practice power and sample size calculations in A/B testing. Moreover, you will explore the impact of early stopping in A/B testing. Finally, you will be introduced to investigating causality through observational studies.

Lab Mechanics

rubric={mechanics:5}

- Paste the URL to your GitHub repo here: **INSERT YOUR GITHUB REPO URL HERE**
- Once you finish the assignment, you must **knit** this `R` markdown to create a `.pdf` file and push everything to your GitHub repo using `git push`. You are responsible for ensuring all the figures, texts, and equations in the `.pdf` file are appropriately rendered.
- **You must submit the rendered `.pdf` file to Gradescope.**

  Heads-up: You need to have a minimum of 3 commits.

Code Quality

rubric={quality:3}

The code that you write for this assignment will be given one overall grade for code quality. Check our **code quality rubric** as a guide to what we are looking for. Also, for this course (and other MDS courses that use `R`), we are trying to follow the `tidyverse` code style. There is a guide you can refer too: http://style.tidyverse.org/

Each code question will also be assessed for code accuracy (i.e., does it do what it is supposed to do?).

Writing

rubric={writing:3}

To get the marks for this writing component, you should:

- Use proper English, spelling, and grammar throughout your submission (the non-coding parts).
- Be succinct. **This means being specific about what you want to communicate, without being superfluous.**

Check our **writing rubric** as a guide to what we are looking for.

A Note on Challenging Questions

Each lab will have a few challenging questions. These are usually low-risk questions and will contribute to maximum 5% of the lab grade. The main purpose here is to challenge yourself or dig deeper in a particular area. When you start working on labs, attempt all other questions before moving to these questions. If you are running out of time, please skip these questions.

We will be more strict with the marking of these questions. If you want to get full points in these questions, your answers need to

- be thorough, thoughtful, and well-written,
- provide convincing justification and appropriate evidence for the claims you make, and
- impress the reader of your lab with your understanding of the material, your analytical and critical reasoning skills, and your ability to think on your own.

Setup

If you fail to load any packages, you can install them and try loading the library again.

```r
library(tidyverse)
library(purrr)
library(kableExtra)
library(cowplot)
```

Exercise 1: Understanding and Calculating Power

We are going to go through the motions of hypothesis testing. This will help us frame our power calculations in this exercise and determine an appropriate sample size in **Exercise 2**.

> **Note:** This refresher on the **Power of the Test** might be helpful to solve this exercise.

Q1.1. Formulating the Hypotheses

rubric={reasoning:4}

Suppose a company's marketing team has developed a new video for their TikTok ad. They want to know if this *new* ad will increase the ad engagement (which they will measure via **ad dwell time** in seconds) compared to the *original* ad they are currently running. Therefore, you have two experimental treatments: *original* and *new* ad.

Write down the hypotheses in this context for a **one-sided** test in plain words **AND** mathematically. **Do not forget to define your mathematical notation.**

ANSWER:

> *Let μ_{new} and $\mu_{original}$ be the mean dwell times of the new and original ads, respectively.*
>
> *H_0: The new TikTok ad has the same mean ad dwell time as the original TikTok ad, i.e.,*
>
> $$\delta = \mu_{\text{new}} - \mu_{\text{original}} = 0.$$
>
> *H_a: The new TikTok ad's dwell time is larger from that of the original TikTok ad, i.e.,*
>
> $$\delta = \mu_{\text{new}} - \mu_{\text{original}} > 0.$$

Q1.2. Computing the Standard Error

rubric={accuracy:1}

The company would like to run an experiment on TikTok users in the age demographic most of their customers fall (between 16 and 24 years old).

Suppose that they have previously collected data on several other video ads from this demographic. These **sample data** estimate that the standard deviation of **any** ad dwell time will be 15 seconds. The distribution of ad dwell time will be **approximately symmetric**.

If we had **100 viewers per treatment**, **what would be the approximate standard error for the estimated mean difference $\hat{\delta} = \hat{\mu}_{\textbf{new}} - \hat{\mu}_{\textbf{original}}$?**

If the population variances are unknown, recall the standard error $\text{SE}\left(\hat{\delta}\right)$ for the difference between two sample means is:

$$\text{SE}\left(\hat{\delta}\right) = \sqrt{\frac{S^2_{\text{new}}}{n_{\text{new}}} + \frac{S^2_{\text{original}}}{n_{\text{original}}}},$$

Assuming that the variances S^2_{new} and S^2_{original} are equal in both treatments, while $n = n_{\text{new}} = n_{\text{original}}$, compute $\text{SE}\left(\hat{\delta}\right)$ and store this value in a vector with the name `se_diff_means_n_100`.

```
se_diff_means_n_100 <- NULL

# BEGIN SOLUTION
se_diff_means_n_100 <- sqrt((15^2 / 100) + (15^2 / 100))
# END SOLUTION
```

```
se_diff_means_n_100
```

```
## [1] 2.12132
```

Q1.3. Plotting the Null Distribution of $\hat{\delta}$

rubric={viz:4}

What does the **theoretical sampling distribution** of the mean difference

$$\hat{\delta} = \hat{\mu}_{\text{new}} - \hat{\mu}_{\text{original}} = \bar{X}_{\text{new}} - \bar{X}_{\text{original}}$$

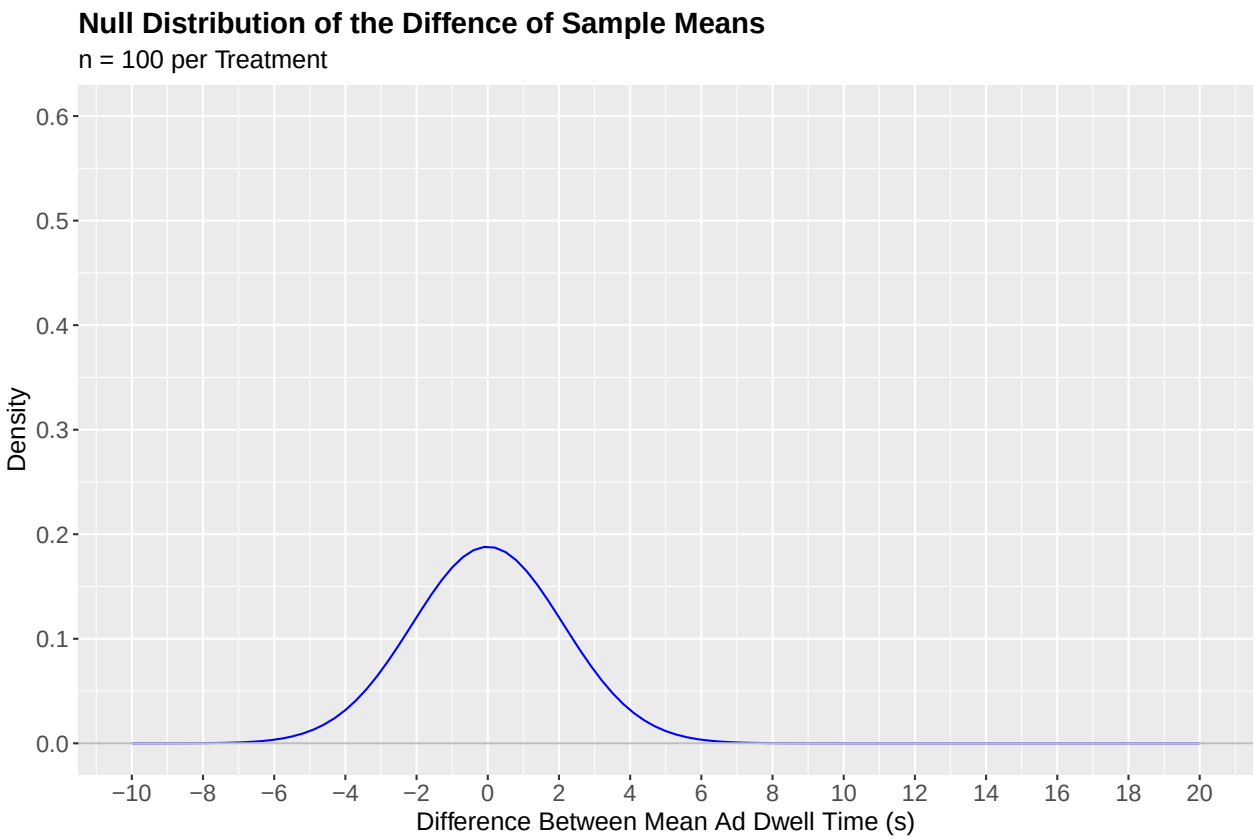
look like when the null hypothesis is true?

Assume the degrees of freedom are greater than 30 in this case, so the distribution of $\hat{\delta}$ will be approximately **Normal** (**recall the behaviour of the t-distribution when the degrees of freedom tend to infinity!**). The standard deviation of this distribution (i.e., the standard error for practical purposes) would be about the value calculated in **Q1.2** (i.e., `se_diff_means_n_100`). **Under the null hypothesis**, the **mean difference** would be equal to `0`.

Plot this distribution in blue where the x-axis is the **difference between mean ad dwell time (in seconds)** and the y-axis is the density of this **theoretical sampling distribution**. Store the plot in `hypoth_diff_dist`.

Set your x-axis limits between `-10` and `20`. Moreover, set the y-axis limits between `0` and `0.6`.

```
hypoth_diff_dist <- NULL

# BEGIN SOLUTION
hypoth_diff_dist <- ggplot(data = data.frame(x = c(-10, 20)), aes(x)) +
  stat_function(
    fun = dnorm,
    args = list(mean = 0, sd = se_diff_means_n_100),
    colour = "blue"
  ) +
  xlab("Difference Between Mean Ad Dwell Time (s)") +
  ylab("Density") +
  geom_hline(yintercept = 0, color = "gray") +
  theme(
    plot.title = element_text(size = 15, face = "bold"),
    plot.subtitle = element_text(size = 13),
    axis.text.x = element_text(size = 12, angle = 0),
    axis.text.y = element_text(size = 12, angle = 0),
    axis.title = element_text(size = 13)
  ) +
  ggtitle(
    label = "Null Distribution of the Diffence of Sample Means",
    subtitle = "n = 100 per Treatment"
  ) +
  scale_x_continuous(breaks = seq(-10, 20, 2)) +
  scale_y_continuous(breaks = seq(0, 0.6, 0.1), limits = c(0, 0.6))
# END SOLUTION

hypoth_diff_dist
```



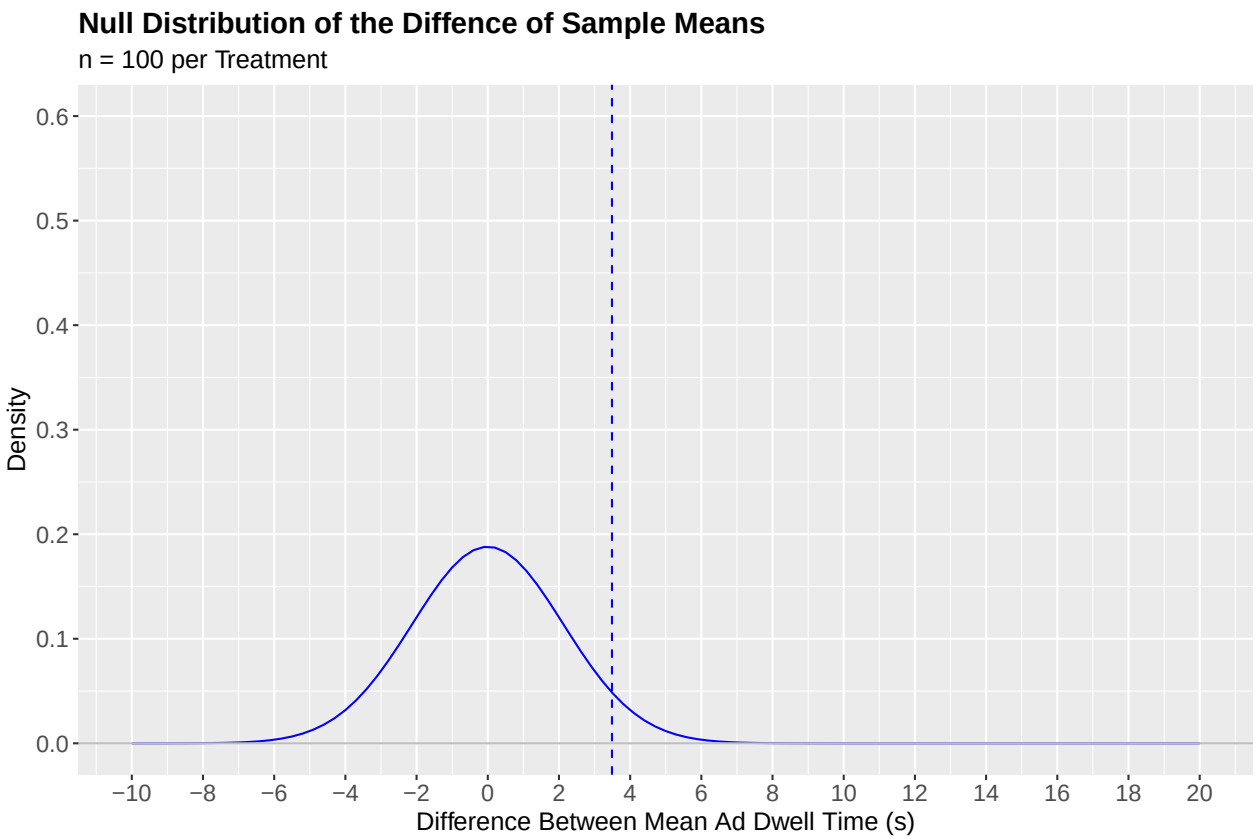
Q1.4. Plotting Critical Value on the Null Distribution

rubric={accuracy:2,viz:3}

For what **critical value** of $\hat{\delta}$ would we reject the null hypothesis if we set the significance level $\alpha = 0.05$?

Draw this value as a boundary (**using a vertical dashed blue line**) on the null distribution of $\hat{\delta}$ you plotted in **Q1.3** (i.e., object `hypoth_diff_dist`).

```
# BEGIN SOLUTION
critical_value_n_100 <- qnorm(0.95, mean = 0, sd = se_diff_means_n_100)

hypoth_diff_dist <- hypoth_diff_dist +
  geom_vline(
    xintercept = critical_value_n_100,
    colour = "blue",
    linetype = "dashed"
  )
hypoth_diff_dist
```




```
# END SOLUTION
```

Q1.5. Plotting the Alternative Distribution of $\hat{\delta}$

rubric={viz:4}

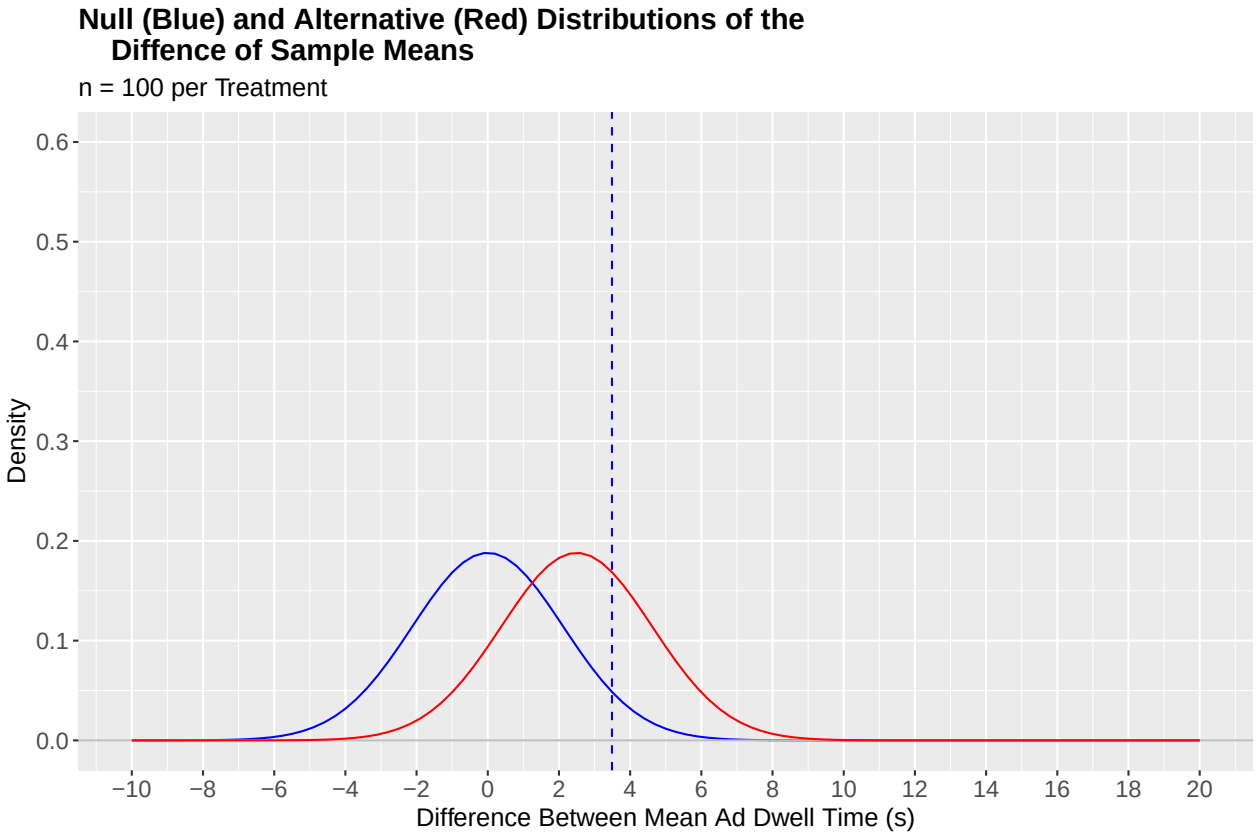
We will perform some hypothetical calculations to determine the probability we reject the null hypothesis if the alternative hypothesis were actually true. This probability is called the **power**, and we can compute it for **different sample sizes** or **different effect sizes**.

The company, in this example, defines a PRACTICAL significant result for a mean ad dwell time difference to be 2.5 seconds. Suppose we proceeded with our experiment as suggested above (**100 experimental units, i.e. viewers, per treatment**), and the new video ad increased the dwell time by an additional 2.5 seconds. We would like to calculate the probability that we would detect that increase, which is the power.

Before we do that calculation, on the object `hypoth_diff_dist`, **include in red the Normal alternative distribution when the mean difference is 2.5 seconds** (assume the standard deviation is equal to `se_diff_means_n_100`).

```
# BEGIN SOLUTION
hypoth_diff_dist <- hypoth_diff_dist +
  stat_function(
    fun = dnorm,
    args = list(mean = 2.5, sd = se_diff_means_n_100),
    colour = "red"
  ) +
  ggtitle(
    label = "Null (Blue) and Alternative (Red) Distributions of the
```

```
    Diffence of Sample Means",
    subtitle = "n = 100 per Treatment"
  )
hypoth_diff_dist
```




```
# END SOLUTION
```

Q1.6. Calculating Power with 100 Experimental Units per Treatment

rubric={accuracy:2}

To calculate the power for observing this **effect size** (i.e., the difference is equal to 2.5 seconds) under the specified experimental conditions (100 experimental units per treatment and assuming the SE $\left(\hat{\delta}\right)$ `se_diff_means_n_100`), we need to find the **distributional area** where the true difference in mean dwell time is equal to 2.5 seconds. This area lies to the right-hand side of the upper 5% boundary (i.e., the vertical dashed blue line) of the distribution of the null hypothesis **but under red the distribution of the alternative hypothesis**. The plot in **Q1.5** will help you to visualize this.

Store this value in a vector with the name `power_n_100`.

```
power_n_100 <- NULL

# BEGIN SOLUTION
power_n_100 <- round(pnorm(critical_value_n_100,
  mean = 2.5, sd = se_diff_means_n_100, lower.tail = FALSE
), 4)
# END SOLUTION
```

```
power_n_100
```

```
## [1] 0.3205
```

Q1.7. Increasing to $1,000$ Experimental Units per Treatment

rubric={viz:8}

Create the same visualization like the one from **Q1.5**, this time assuming 1000 ad viewers (i.e., experimental units) per treatment. Again, assume that the variances S^2_{new} and S^2_{original} are equal between both treatments i.e., 15^2 seconds^2. Include proper axes labels.

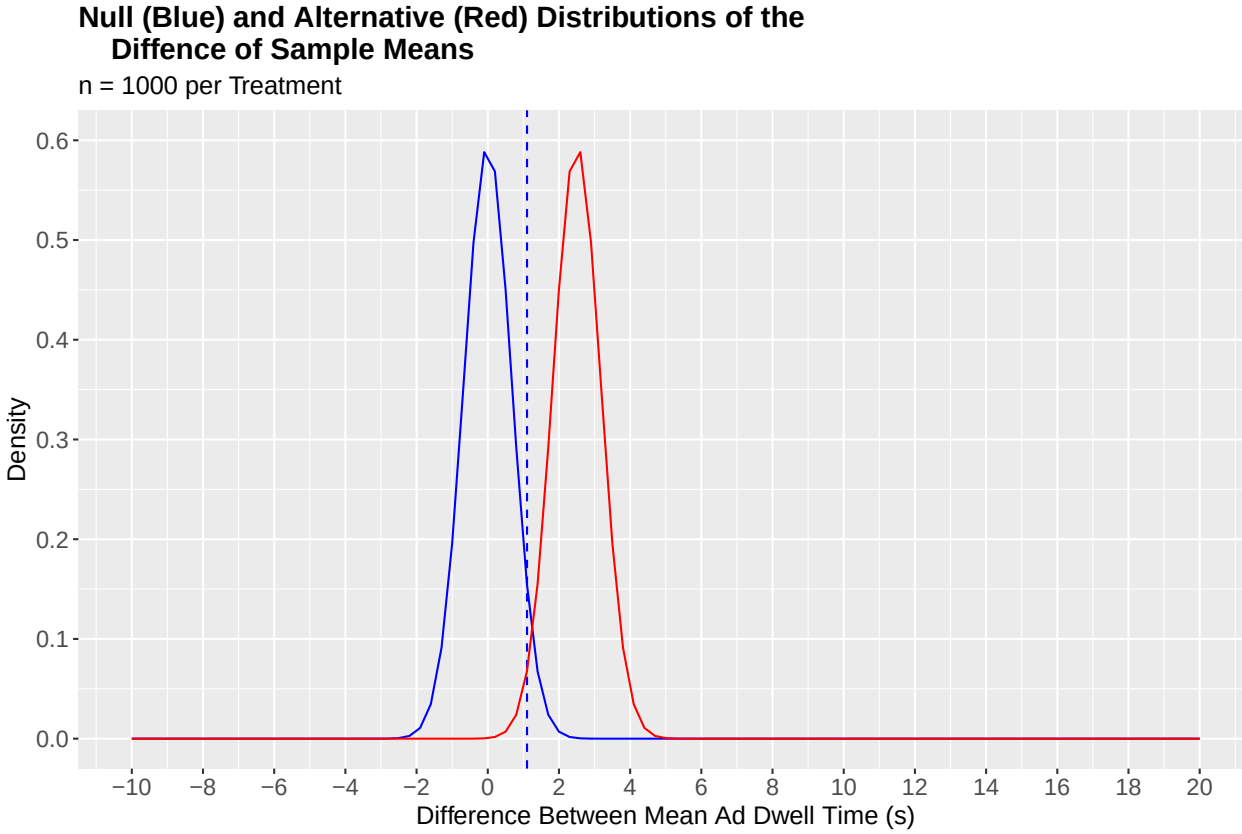
Assign the name `hypoth_diff_dist_1000` to your plot. Set your x-axis limits between `-10` and `20`. Moreover, set the y-axis limits between `0` and `0.6`.

> **Hint:** You will need to compute a new standard error:

$$\text{SE}\left(\hat{\delta}\right) = \sqrt{\frac{S^2_{\text{new}}}{n_{\text{new}}} + \frac{S^2_{\text{original}}}{n_{\text{original}}}}.$$

```
hypoth_diff_dist_1000 <- NULL

# BEGIN SOLUTION
se_diff_means_n_1000 <- sqrt((15^2 / 1000) + (15^2 / 1000))
critical_value_n_1000 <- qnorm(0.95, mean = 0, sd = se_diff_means_n_1000)

hypoth_diff_dist_1000 <-ggplot(data = data.frame(x = c(-10, 20)), aes(x)) +
  stat_function(
    fun = dnorm,
    args = list(mean = 0, sd = se_diff_means_n_1000),
    colour = "blue"
  ) +
  geom_vline(
    xintercept = critical_value_n_1000,
    colour = "blue",
    linetype = "dashed"
  ) +
  stat_function(
    fun = dnorm,
    args = list(mean = 2.5, sd = se_diff_means_n_1000),
    colour = "red"
  ) +
  ggtitle(
    label = "Null (Blue) and Alternative (Red) Distributions of the
    Diffence of Sample Means",
    subtitle = "n = 1000 per Treatment"
  ) +
  scale_x_continuous(breaks = seq(-10, 20, 2)) +
  scale_y_continuous(breaks = seq(0, 0.6, 0.1), limits = c(0, 0.6)) +
  xlab("Difference Between Mean Ad Dwell Time (s)") +
  ylab("Density") +
  theme(
    plot.title = element_text(size = 15, face = "bold"),
    plot.subtitle = element_text(size = 13),
    axis.text.x = element_text(size = 12, angle = 0),
```

```
    axis.text.y = element_text(size = 12, angle = 0),
    axis.title = element_text(size = 13)
  )
# END SOLUTION

hypoth_diff_dist_1000
```



Q1.8. Calculating Power with $1,000$ Experimental Units per Treatment

rubric={accuracy:2}

Calculate the power for an experiment using this larger sample size ($n = 1,000$ per treatment).

Store this value in a vector with the name `power_n_1000`.

```
power_n_1000 <- NULL

# BEGIN SOLUTION
power_n_1000 <- round(pnorm(critical_value_n_1000,
  mean = 2.5, sd = se_diff_means_n_1000, lower.tail = FALSE
), 4)
# END SOLUTION

power_n_1000
```

```
## [1] 0.9813
```

Q1.9. Final Comments on Increasing the Sample Size

rubric={reasoning:4}

In **Q1.8**, we should observe a very high power for a sample size of $n = 1000$ for each treatment Could there be any **practical downside** to running an experiment with this sample size?

If there is a **practical downside**, what might we prefer to do instead of running an experiment under these conditions? On the other hand, if there is no practical downside and this is ideal, why is that?

> **Note:** When thinking about practical downsides, consider factors out of the statistical framework (e.g., business terms).

Answer in three or four sentences.

ANSWER:

> *Yes, there could be a practical downside to running this experiment with $n = 1000$ for each treatment. Although we are very likely to observe an effect if the alternative hypothesis is true under these conditions, we risk showing a potentially worse ad to many viewers. This could lead to decreased downstream financial consequences (e.g., fewer people learning about the company/product shown in the ad, fewer people visiting the company's website, fewer people buying the product/subscription, etc). We would prefer to find a smaller sample size that still gives us high power. Hence, if one of the ads is less engaging, we will show it to as few viewers as possible while we learn this.*

Exercise 2: What Sample Size to Use

In **Exercise 1**, we explored how we can calculate power given a sample size. Frequently, we want to do the opposite, calculate a sample size for a given power. This is useful for planning experiments. We do this by flipping the math in the equations of the hypothesis testing.

Q2.1. First Manual Sample Size Computation per Treatment

rubric={accuracy:3}

Let us calculate the sample size **per treatment** where our power is 90% using the case from **Exercise 1** with $\alpha = 0.05$. Do this using `R` as a manual calculator (i.e., **do not use the function from the `pwr` package**). Again, assume that the population standard deviations are equal in both treatments, i.e., approximately 15 seconds.

The sample size computation formulae vary depending on the hypothesis test (or even on a given regression model!). For this test on the continuous means of two independent populations, the formula for the **sample size n PER TREATMENT** is:

$$n = \frac{2S_p^2(Z_{1-\alpha} + Z_{1-\beta})^2}{\texttt{diff}^2},$$

where α is the desired significance level, $1-\beta$ is the desired power, `diff` is the hypothesized mean difference (2.5 seconds), and the pooled variance is

$$S_p^2 = \frac{S_{\text{new}}^2 + S_{\text{original}}^2}{2}.$$

> **Heads-up:** $Z_{1-\alpha}$ and $Z_{1-\beta}$ are quantiles coming from a Standard Normal distribution (i.e., a Normal distribution with mean equal to `0` and standard deviation equal to `1`).

Store this value in a vector with the name `tiktok_sample_size`.

```
tiktok_sample_size <- NULL

# BEGIN SOLUTION
# Getting upper tails of 95% and 90%.
z_score_0.95 <- qnorm(0.05, lower.tail = FALSE)
z_score_0.90 <- qnorm(0.10, lower.tail = FALSE)

tiktok_sample_size <- (2 * 15^2 * (z_score_0.95 + z_score_0.90)^2) / (2.5)^2
tiktok_sample_size <- ceiling(tiktok_sample_size)
# END SOLUTION

tiktok_sample_size
```

```
## [1] 617
```

Q2.2. Second Manual Sample Size Computation per Treatment

rubric={accuracy:3}

Continuing with the TikTok ad example we have been working with so far. Suppose the targeted power was 95%, and we were using $\alpha = 0.01$. **What sample size *per treatment* should we use for these experimental conditions?** Use `R` as a **manual calculator**.

Store this value in a vector with the name `tiktok_sample_size_stricter`.

```r
tiktok_sample_size_stricter <- NULL

# BEGIN SOLUTION
# Getting upper tail of 99% and 95%.
z_score_0.99 <- qnorm(0.01, lower.tail = FALSE)
z_score_0.95 <- qnorm(0.05, lower.tail = FALSE)

tiktok_sample_size_stricter <- (2 * 15^2 * (z_score_0.99 + z_score_0.95)^2) / (2.5)^2
tiktok_sample_size_stricter <- ceiling(tiktok_sample_size_stricter)
# END SOLUTION

tiktok_sample_size_stricter
```

```
## [1] 1136
```

Q2.3. Sample Size Computation per Treatment using Package `pwr`

rubric={accuracy:2}

Now use the function from the `pwr` `R` package to perform two sample size estimations for the TikTok ad experiment **per treatment**, by setting:

1. Power as 90% and $\alpha = 0.05$.
2. Power as 95% and $\alpha = 0.01$.

The output has to be **practically equal** to your results in **Q2.1** and **Q2.2**.

```r
# BEGIN SOLUTION
library(pwr)

power_test_1 <- pwr.t.test(
  d = 2.5 / 15,
  sig.level = 0.05,
  power = 0.90,
  type = "two.sample",
  alternative = "greater"
)
power_test_1
```

```
##
##      Two-sample t test power calculation
##
##               n = 617.2746
##               d = 0.1666667
##       sig.level = 0.05
##           power = 0.9
##     alternative = greater
##
## NOTE: n is number in *each* group
```

```r
power_test_2 <- pwr.t.test(
  d = 2.5 / 15,
  sig.level = 0.01,
  power = 0.95,
  type = "two.sample",
  alternative = "greater"
)
power_test_2
```

```
##
##      Two-sample t test power calculation
##
##               n = 1136.826
##               d = 0.1666667
##       sig.level = 0.01
##           power = 0.95
##     alternative = greater
##
## NOTE: n is number in *each* group
```

```r
# END SOLUTION
```

(Challenging) Q2.4. A More Comprehensive Power Analysis Simulation

rubric={accuracy:3,viz:3}

Simulation is a powerful tool for sample size calculations. When we move to more complex analyses, it is tough to do these sorts of calculations using math. Thus in practice, we very often resort to simulation.

We will again calculate the sample size for the TikTok video example problem where power is 90% and $\alpha = 0.05$. However, instead of relying on mathematical formulas to calculate this, we will use simulation.

To get you started, we give you the code for a function named `t_test`, which takes a sample size (**per treatment**), a **raw effect size** (*not adjusted by the population standard deviation*), and a population standard deviation. It uses these values to create two samples, each drawn from Normal distributions of a sample size specified by the user. The Normal distribution (that the samples are drawn from) corresponds to:

1. What we expect if the null hypothesis is true (mean equal to 0, and standard deviation specified by the user).
2. What we expect if the alternative hypothesis is true (mean is set to the raw effect size specified by the user, along with the standard deviation).

The function then performs a t-test and returns a one-row tibble that has three columns:

- `t_stat`: t-statistic.
- `p_value`: p-value from performing a t-test.
- `significant`: a Boolean which is TRUE when the $p-\text{value} < \alpha$, whose default value is 0.05.

```r
# Run this chunk of code before proceeding.

# Two-sample t-test for simulating power and sample size
# derived
# from: https://cran.r-project.org/web/packages/paramtest/vignettes/Simulating-Power.html

# @param n (numeric): Sample size for each treatment.
# @param effect_size (numeric): Expected raw difference in sample means .
# (when null hypothesis is true versus when alternative hypothesis is true).
# @param pop_sd (numeric): Population standard deviation.
# @param alpha (numeric): Significance threshold (default is 0.05).

# @return tibble: A tibble that has 3 columns; t_stat (t-statistic),
# p_value (p-value from performing a two-sample t-test),
# and significant (a Boolean which is TRUE when
# the p-value was < alpha - whose default value is 0.05).

t_test <- function(n, effect_size = 0, pop_sd, alpha = 0.05) {
  sample_original <- rnorm(n, mean = 0, sd = pop_sd)
  sample_new <- rnorm(n, mean = effect_size, sd = pop_sd)

  # Note the difference is set up to zero by default given that our
  # simulated data is already set up with mean = effect_size.
  # The simulation is designed to catch any signal different from zero.
  t_test_results <- t.test(
    x = sample_new, y = sample_original,
    var.equal = TRUE,
    alternative = "greater"
  )
  t_stat <- t_test_results$statistic
  p_value <- t_test_results$p.value
```

```r
  return(tibble(t_stat, p_value, significant = (p_value < alpha)))
}

# Example
set.seed(554)
t_test(n = 1000, effect_size = 2, pop_sd = 5)
```

```
## # A tibble: 1 x 3
##   t_stat  p_value significant
##    <dbl>    <dbl> <lgl>
## 1   7.79 5.18e-15 TRUE
```

Your task is to extend this code to do the following:

1. Write another function which calls the `t_test` function many times (say 1000 replicates) and calculates the power. Call it `t_test_power`.

```r
t_test_power <- NULL

# BEGIN SOLUTION
t_test_power <- function(n, effect_size, pop_sd, reps = 1000) {
  pmap_df(list(rep(n, reps), rep(effect_size, reps), rep(pop_sd, reps)), t_test) %>%
    summarise(power = mean(significant)) %>%
    pull()
}
# END SOLUTION

t_test_power
```

```
## function(n, effect_size, pop_sd, reps = 1000) {
##   pmap_df(list(rep(n, reps), rep(effect_size, reps), rep(pop_sd, reps)), t_test) %>%
##     summarise(power = mean(significant)) %>%
##     pull()
## }
```

2. Calculate the power for the TikTok video example (where $\alpha = 0.05$) from $n = 100$ to 1000 increasing by 50 experimental units per treatment. Recall that the raw effect size is 2.5 seconds, and the population standard deviation is 15 seconds for both treatments.

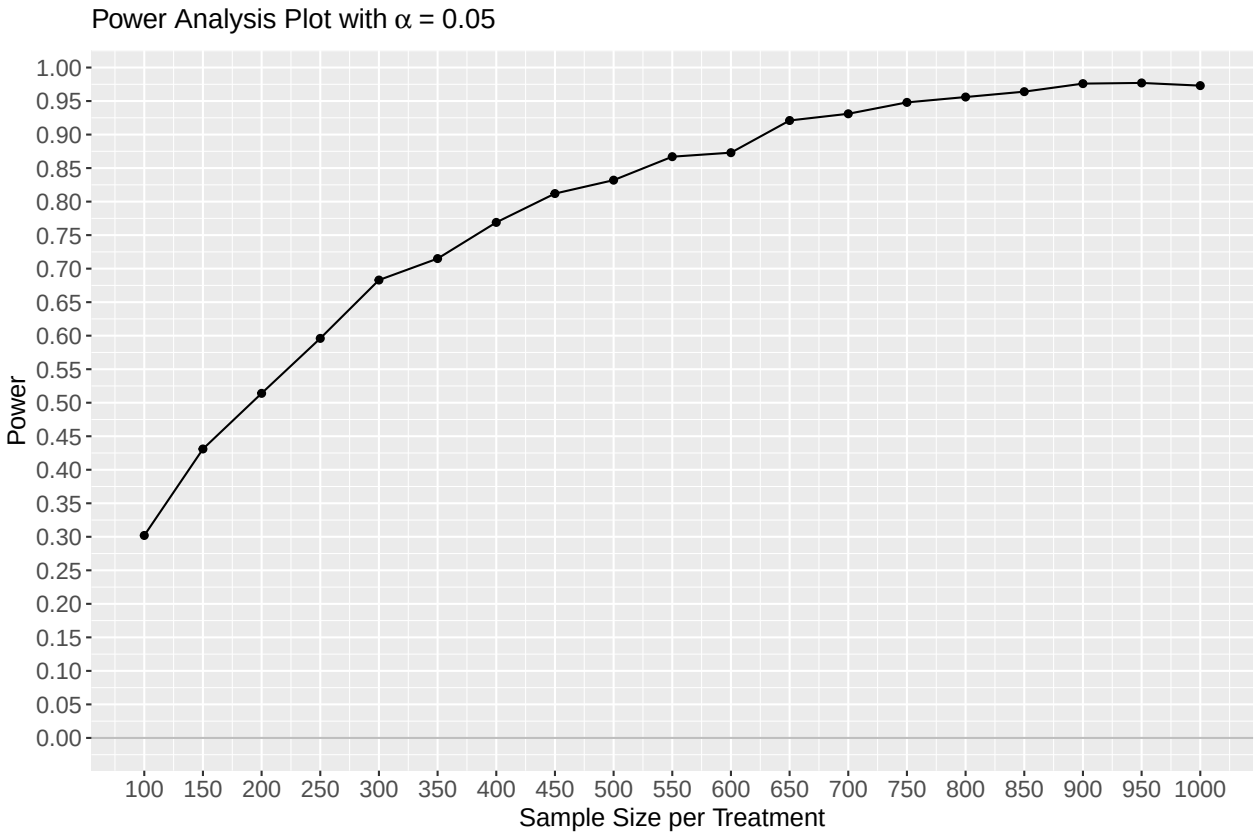
```r
# Do not delete this line.
set.seed(123)

# BEGIN SOLUTION
sample_size_power <- tibble(
  sample_size = seq(100, 1000, 50),
  power = seq_along(sample_size)
)

for (i in 1:nrow(sample_size_power)) {
  sample_size_power$power[i] <- t_test_power(sample_size_power$sample_size[i],
    effect_size = 2.5,
    pop_sd = 15
  )
}

sample_size_power
```

```
## # A tibble: 19 x 2
##    sample_size power
##          <dbl> <dbl>
##  1         100 0.302
##  2         150 0.431
##  3         200 0.514
##  4         250 0.596
##  5         300 0.683
##  6         350 0.715
##  7         400 0.769
##  8         450 0.812
##  9         500 0.832
## 10         550 0.867
## 11         600 0.873
## 12         650 0.921
## 13         700 0.931
## 14         750 0.948
## 15         800 0.956
## 16         850 0.964
## 17         900 0.976
## 18         950 0.977
## 19        1000 0.973
```

```
# END SOLUTION
```

3. Plot the sample size (x-axis) versus power (y-axis). Include proper axes labels and title.

```
# BEGIN SOLUTION
sample_size_power_plot <- sample_size_power %>%
  ggplot(aes(x = sample_size, y = power)) +
  geom_point() +
  geom_line() +
  ylab("Power") +
  xlab("Sample Size per Treatment") +
  scale_y_continuous(breaks = seq(0, 1, 0.05)) +
  scale_x_continuous(breaks = seq(0, 1000, 50)) +
  theme(
    plot.title = element_text(size = 13.5),
    axis.text.x = element_text(size = 12, angle = 0),
    axis.text.y = element_text(size = 12, angle = 0),
    axis.title = element_text(size = 13)
  ) +
  geom_hline(yintercept = 0, color = "gray") +
  ggtitle(expression("Power Analysis Plot with"~alpha~"= 0.05"))

sample_size_power_plot
```




```
# END SOLUTION
```

Exercise 3: Basics of Early Stopping in Hypothesis Testing

As in **Q2.4**, we will work with different simulations to better understand the implications on early stopping in A/B testing. Let us retake the example of the TikTok ad. Recall they want to know if a *new* ad will cause an increase in the ad engagement (which they will measure via **ad dwell time**) compared to the *original* ad currently running. To do this, they will run an A/B testing.

Suppose that initially they plan to run the experiment with the **original** sample size per group $n = n_{\text{original}} = n_{\text{new}}$. Nonetheless, in terms of early stopping, the team would collect the data **gradually** for both groups and they will stop the experiment **once they find a significant result** (i.e., $p - \text{value} < \alpha$).

First, let us see how the p-values evolve as we increase the sample size until getting to n without the early stopping **in a single A/B testing**. The function `incremental_t_test()` performs a two-sample t-test beginning a given `increasing_size` for both groups (**before the `for` loop, it is set up equal to `sample_increase`**), then it gradually increases by `sample_increase` until getting to `n` for a single experiment. It returns a tibble that has two columns:

- `inc_sample_size`: the final sample size per group for which the t-test is performed.
- `p_value`: p-value from performing a given t-test.

```r
# Two-sample t-test for tracking p-values by incremental sample sizes until getting to n.

# @param n (numeric): Initially planned sample size for each treatment,
# n = n_original = n_new.
# @param effect_size (numeric): Difference in sample means expected (when null hypothesis
# is true versus when alternative
# hypothesis is true).
# @param pop_sd (numeric): Population standard deviation.
# @param sample_increase (numeric): Initial sample size in A/B testing and sample
# size increment.
# @param alpha (numeric): Significance threshold (default is 0.05).

# @return p.value.df: A tibble that has 2 columns; inc_sample_size
# (increasing sample size)
# and p_value (p-value from performing a two-sample t-test).

incremental_t_test <- function(n, effect_size, pop_sd, sample_increase, alpha = 0.05) {
  sample_original <- rnorm(n, mean = 0, pop_sd)
  sample_new <- rnorm(n, mean = effect_size, pop_sd)

  p.value.df <- tibble(
    inc_sample_size = rep(0, n / sample_increase),
    p_value = rep(0, n / sample_increase)
  )

  increasing_size <- sample_increase

  for (i in 1:nrow(p.value.df))
  {
    # Note the difference is set up to zero by default given that our
    # simulated data is already set up with mean = effect_size.
    # The simulation is designed to catch any signal different from zero.
    t_test_results <- t.test(
      x = sample_new[1:increasing_size],
      y = sample_original[1:increasing_size],
      var.equal = TRUE,
```

```
      alternative = "greater"
    )
    p.value.df[i, "p_value"] <- as_tibble(t_test_results$p.value)
    p.value.df[i, "inc_sample_size"] <- increasing_size
    increasing_size <- increasing_size + sample_increase
  }

  return(p.value.df)
}
```

Q3.1. p-value Progress in Sequential Two-sample t-tests

rubric={accuracy:1,viz:3,reasoning:3}

Suppose that **the null hypothesis is true in fact** and there is no difference in dwell time between both ads. Assuming that both treatment groups have the same population standard deviation of 15 seconds, with a final sample size per treatment `n` of `100` and $\alpha = 0.05$, use `incremental_t_test()` to track the p-values of sequential two-sample t-tests (beginning a sample size of `2` until `100`).

Then, plot this p-value sequence as a line with the incremental sample size on the x-axis and p-value on the y-axis. Add a dashed horizontal red line that indicates the threshold of $\alpha = 0.05$.

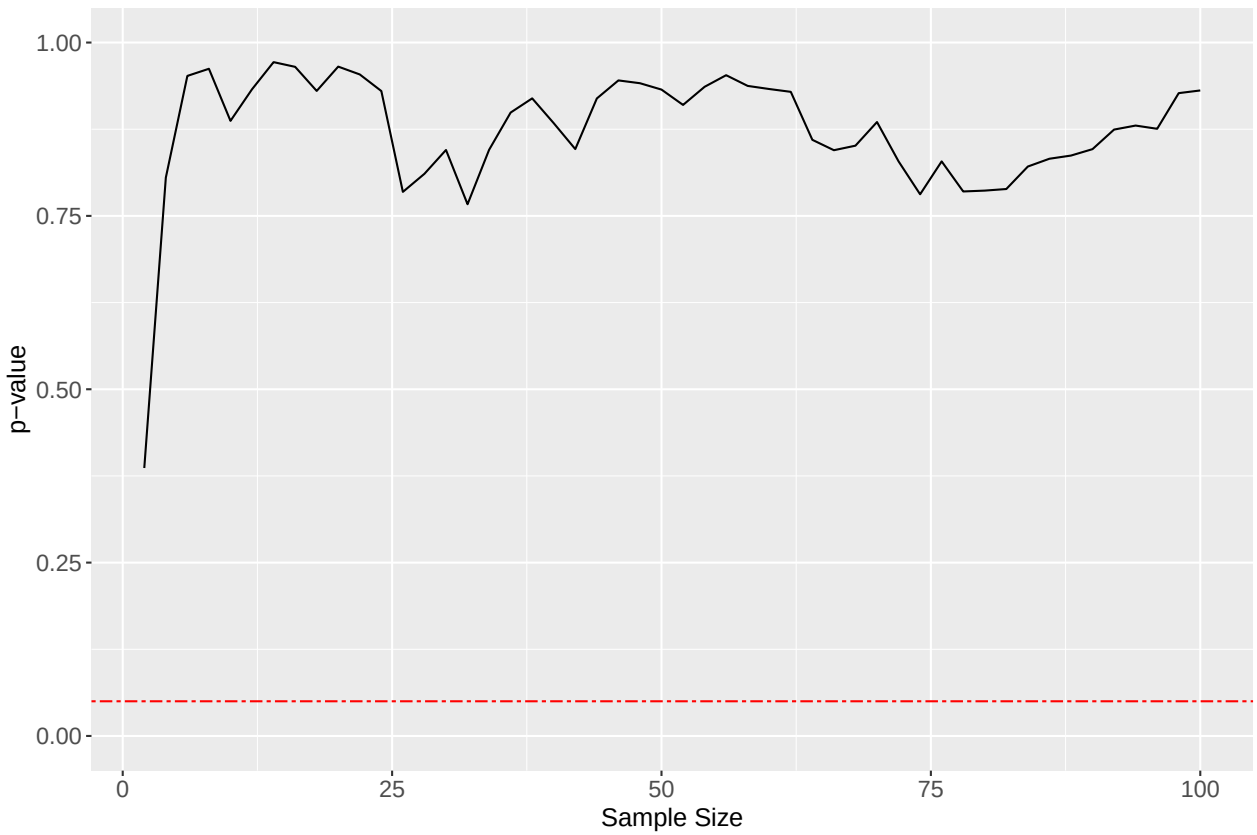
Also, **in one or two sentences**, describe the p-value behaviour.

ANSWER:

> *We can see that this specific simulated A/B testing never rejects the null hypothesis as we increase the sample size we gradually collect. However, it is a really erratic behaviour that does not appear to follow any trend.*

```
# Do not delete this line.
set.seed(123)

# BEGIN SOLUTION
inc_p_values_single_test <- incremental_t_test(
  n = 100, effect_size = 0,
  pop_sd = 15, sample_increase = 2
)

p_value_evolution_plot <- inc_p_values_single_test %>%
  ggplot(aes(x = inc_sample_size, y = p_value)) +
  geom_line() +
  ylab("p-value") +
  xlab("Sample Size") +
  geom_hline(
    yintercept = 0.05,
    colour = "red",
    linetype = "twodash"
  ) +
  coord_cartesian(ylim = c(0, 1)) +
  theme(
    plot.title = element_text(size = 13.5, face = "bold"),
    axis.text.x = element_text(size = 12, angle = 0),
    axis.text.y = element_text(size = 12, angle = 0),
    axis.title = element_text(size = 13)
  )
```

```
p_value_evolution_plot
```




```
# END SOLUTION
```

Q3.2. No Early Stopping when the Null Hypothesis is True

rubric={accuracy:2,viz:3,reasoning:3}

Before going into a full simulation involving A/B testings with early stopping, **let us check what happens when we run the full experiment with the initially planned sample size per treatment**. This will help us to revisit the theoretical concept of the significance level α.

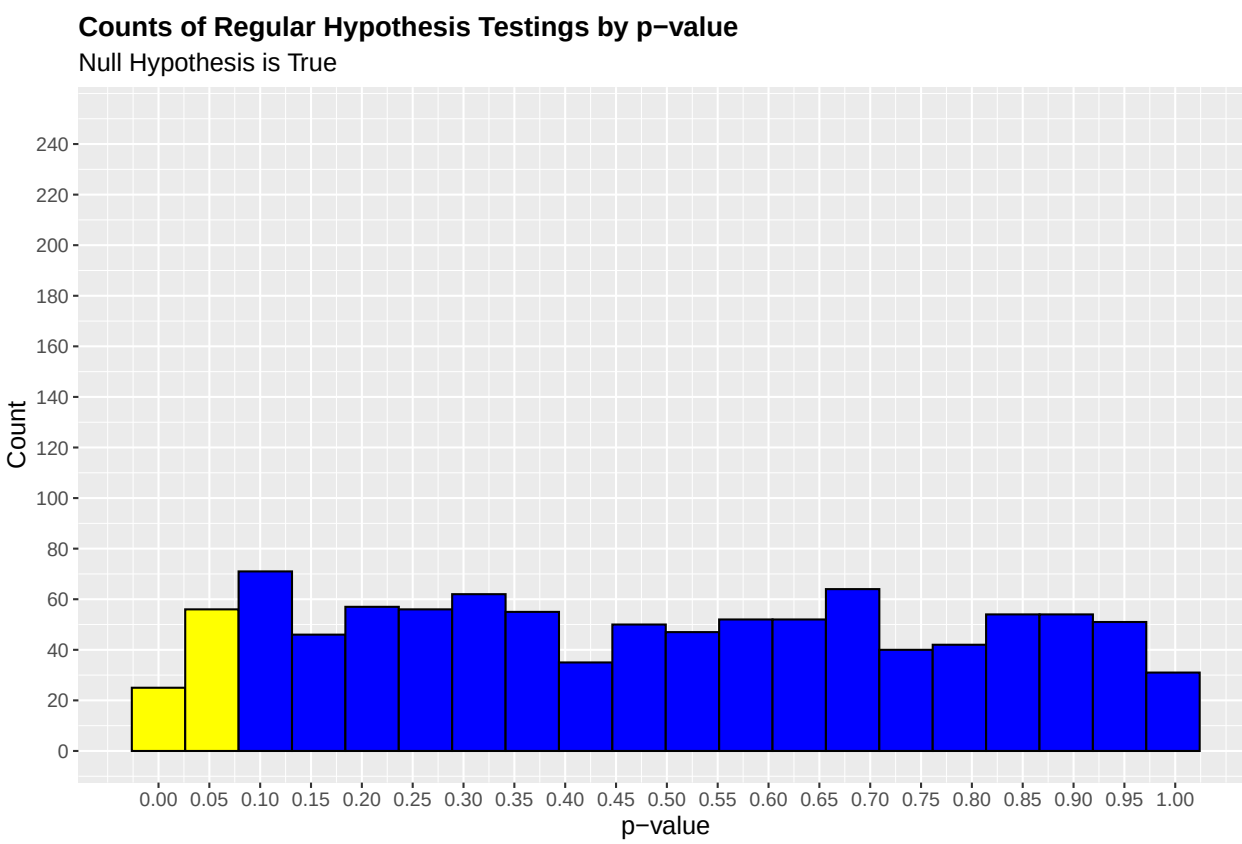
Suppose that **the null hypothesis is true in fact** and there is no difference in dwell time between both ads. Moreover, assume that both treatment groups have the same population standard deviation of 15 seconds. Using the function `t_test()` from **Q2.4** and $\alpha = 0.05$, simulate 1000 A/B testings (each with a sample size of 100 per treatment) and store the results in an data frame called `p_values_reg_test`.

> **Heads-Up:** You do not have to do the challenging exercise **Q2.4** to complete this task. You only need to properly use the function `t_test()` provided there.

Plot the counts of those 1000 A/B testings in a histogram of 20 bins using `p_values_reg_test` with counts on the y-axis and p-value on the x-axis. Moreover, colour the first two bins (from left to right) in yellow and the rest in blue.

```
# Do not delete this line.
set.seed(554)

p_values_reg_test <- NULL
```

```r
# BEGIN SOLUTION
p_values_reg_test <- pmap_df(list(rep(100, 1000), rep(0, 1000), rep(15, 1000)), t_test)

count_hist_p_values_alpha <- ggplot(p_values_reg_test, aes(x = p_value)) +
  geom_histogram(bins = 20, color = "black", fill = c(
    rep("yellow", 2),
    rep("blue", 18)
  )) +
  coord_cartesian(ylim = c(0, 250)) +
  scale_x_continuous(breaks = seq(0, 1, 0.05)) +
  scale_y_continuous(breaks = seq(0, 240, 20)) +
  xlab("p-value") +
  ylab("Count") +
  theme(
    plot.title = element_text(size = 13.5, face = "bold"),
    plot.subtitle = element_text(size = 13),
    axis.text.x = element_text(size = 10, angle = 0),
    axis.text.y = element_text(size = 10, angle = 0),
    axis.title = element_text(size = 13)
  ) +
  ggtitle("Counts of Regular Hypothesis Testings by p-value",
    subtitle = "Null Hypothesis is True"
  )
count_hist_p_values_alpha
```




```r
# END SOLUTION
```

Finally, from `p_values_reg_test`, obtain the proportion of those 1000 A/B tests with `TRUE` in the column

`significant` (compute it in the below code chunk).

```
# BEGIN SOLUTION
empirical_alpha_reg <- p_values_reg_test %>%
  summarise(alpha = mean(significant))
empirical_alpha_reg
```

```
## # A tibble: 1 x 1
##   alpha
##   <dbl>
## 1  0.05
```

```
# END SOLUTION
```

Answer the following in one or two sentences per item:

1. What does this proportion represent in your simulation in terms of the hypothesis testing?
2. Is it related to a fundamental concept in hypothesis testing? Which fundamental concept is it?
3. If it is related to a fundamental concept in hypothesis testing, what testing parameter should this proportion converge to?

ANSWER:

1. *The value 0.05 represents the proportion of those A/B testings where the null hypothesis (i.e., the two ad dwell times are equal) was rejected,* ***even though it is true in fact****.*
2. *Yes, this is committing Type I error.*
3. *We can see that this empirical proportion should converge to the significance level α, our maximum level of tolerance for the Type I error. Note that theoretically speaking with $\alpha = 0.05$, we expect around 5% of the experiments to commit Type I error in the long run.*

Q3.3. Early Stopping when the Null Hypothesis is True

rubric={accuracy:2,viz:3,reasoning:3}

Now, let us perform a similar simulation as in **Q3.2** but with early stopping (**as fully described at the beginning of this Exercise 3**). The function `early_stopping_t_test()` performs a two-sample t-test beginning a given `sample_increase` for both treatments **in a single A/B testing**. The function gradually increases `sample_increase` by 2 and it stops once it finds a significant result (i.e., $p-\text{value} < \alpha$).

> **Heads-up**: Note that the `for` loop could get up to the last experimental replicate without reaching significance.

The function returns a one-row tibble that has four columns:

- `sample_increase`: the sample size reached by the A/B testing per treatment.
- `t_stat`: t-statistic from the last t-test in the `for` loop.
- `p_value`: p-value from performing the last t-test in the `for` loop.
- `significant`: a Boolean which is `TRUE` when the $p-\text{value} < \alpha$, whose default value is `0.05`.

```
# Two-sample t-test for simulating power and sample size with early stopping.

# @param n (numeric): Initially planned sample size for each treatment,
# n = n_original = n_new.
# @param effect_size (numeric): Difference in sample means expected (when null hypothesis
# is true versus when alternative hypothesis is true).
# @param pop_sd (numeric): Population standard deviation.
# @param sample_increase (numeric): Initial sample size in A/B testing and
# sample size increment.
# @param alpha (numeric): Significance threshold (default is 0.05).
```

```r
# @return tibble: A tibble that has 4 columns; sample_increase (final sample size per
# treatment coming from the for loop), t_stat (t-statistic), p_value (p-value
# from performing a two-sample t-test), and significant (a Boolean which is
# TRUE when the p-value was < alpha - whose default value is 0.05).

early_stopping_t_test <- function(n, effect_size, pop_sd, sample_increase, alpha = 0.05) {
  sample_original <- rnorm(n, mean = 0, sd = pop_sd)
  sample_new <- rnorm(n, effect_size, sd = pop_sd)

  increasing_size <- sample_increase

  for (i in 1:(n / sample_increase))
  {
    # Note the difference is set up to zero by default given that our
    # simulated data is already set up with mean = effect_size.
    # The simulation is designed to catch any signal different from zero.
    t_test_results <- t.test(
    x = sample_new[1:increasing_size],
    y = sample_original[1:increasing_size],
    var.equal = TRUE,
    alternative = "greater"
  )
    t_stat <- t_test_results$statistic
    p_value <- t_test_results$p.value
    if (p_value < alpha) break
    increasing_size <- increasing_size + sample_increase
  }

  increasing_size <- increasing_size - sample_increase

  return(tibble(increasing_size, t_stat, p_value, significant = (p_value < alpha)))
}

# Example
# We can see this A/B testing stopped at replicate 64 (in both treatments)
# once the t-test reached significance.
set.seed(123)
early_stopping_t_test(
  n = 100, effect_size = 7,
  pop_sd = 15, sample_increase = 2
)
```

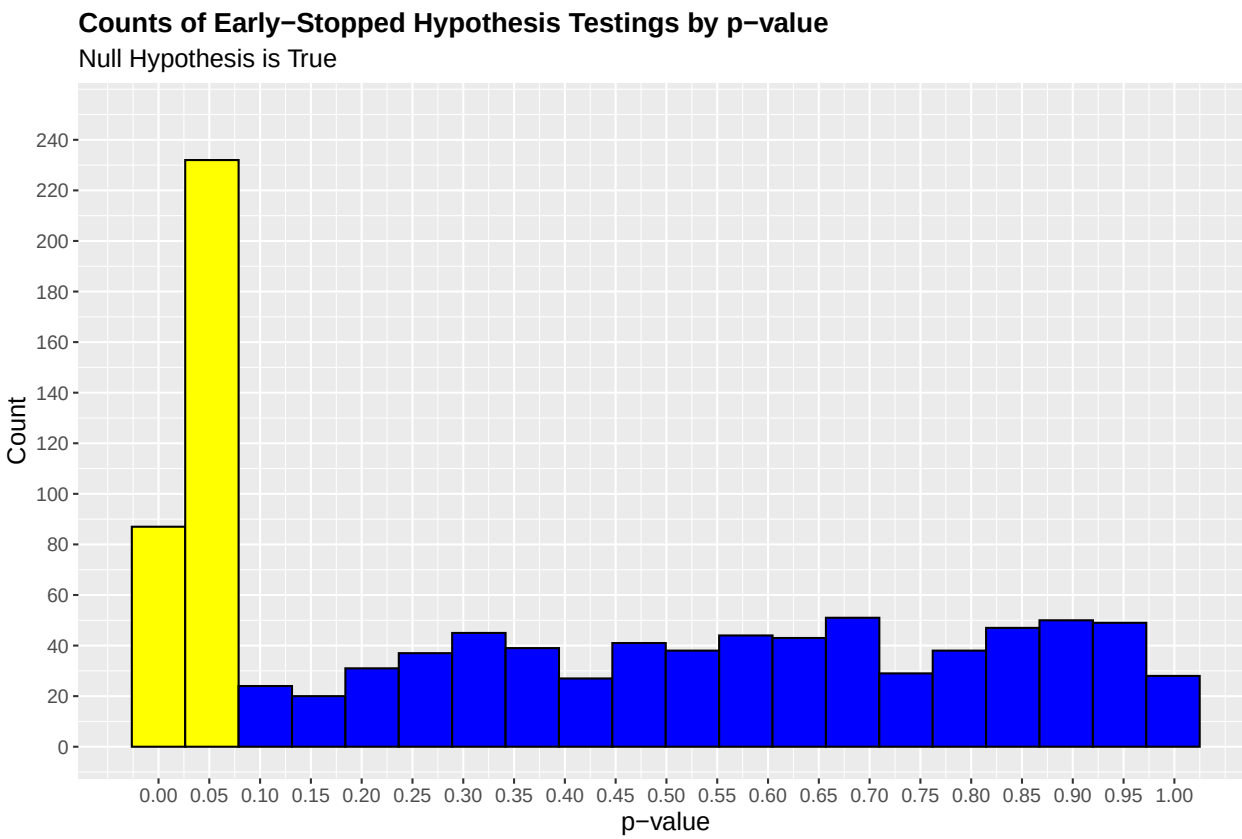
```
## # A tibble: 1 x 4
##   increasing_size t_stat p_value significant
##             <dbl>  <dbl>   <dbl> <lgl>
## 1              64   1.77  0.0396 TRUE
```

Suppose that **the null hypothesis is true in fact** and **there is no difference in dwell time between both ads**. Moreover, assume that both treatment groups have the same population standard deviation of 15 seconds. Increase your group sample by 2 replicates, beginning 2 with a final sample size of 100 elements per treatment. To do this, use the function `early_stopping_t_test()` and $\alpha = 0.05$, simulate 1000 A/B testings and store the results in an data frame called `p_values_early_test`.

Plot the counts of those 1000 A/B testings in a histogram of 20 bins using `p_values_early_test` with counts on the y-axis and p-value on the x-axis. Moreover, colour the first two bins (from left to right) in

yellow and the rest in blue.

```r
# Do not delete this line.
set.seed(554)

p_values_early_test <- NULL

# BEGIN SOLUTION
p_values_early_test <- pmap_df(
  list(rep(100, 1000), rep(0, 1000), rep(15, 1000), rep(2, 1000)),
  early_stopping_t_test
)

count_hist_p_values_early_alpha <- ggplot(p_values_early_test, aes(x = p_value)) +
  geom_histogram(bins = 20, color = "black", fill = c(rep("yellow", 2),
                                                      rep("blue", 18))) +
  coord_cartesian(ylim = c(0, 250)) +
  scale_x_continuous(breaks = seq(0, 1, 0.05)) +
  scale_y_continuous(breaks = seq(0, 240, 20)) +
  xlab("p-value") +
  ylab("Count") +
  theme(
    plot.title = element_text(size = 13.5, face = "bold"),
    plot.subtitle = element_text(size = 13),
    axis.text.x = element_text(size = 10, angle = 0),
    axis.text.y = element_text(size = 10, angle = 0),
    axis.title = element_text(size = 13)
  ) +
  ggtitle("Counts of Early-Stopped Hypothesis Testings by p-value",
    subtitle = "Null Hypothesis is True"
  )
count_hist_p_values_early_alpha
```




```
# END SOLUTION
```

Finally, from `p_values_early_test`, obtain the proportion of those 1000 A/B tests with `TRUE` in the column `significant` (compute it in the below code chunk).

```
# BEGIN SOLUTION
empirical_alpha_early <- p_values_early_test %>%
  summarise(alpha = mean(significant))
empirical_alpha_early
```

```
## # A tibble: 1 x 1
##   alpha
##   <dbl>
## 1 0.316
```

```
# END SOLUTION
```

Answer the following in two or three sentences per item:

1. How does this proportion compare to the one in **Q3.2**?
2. What does this fact in the previous item (1) indicate?

ANSWER:

1. *The value 0.316 represents the proportion of those A/B testings where the null hypothesis (i.e., the two ad dwell times are equal) was rejected,* ***even though it is true in fact****. It is 6.32 times larger than the one obtained in* ***Q3.2****.*
2. *We can see that early stopping in this framework severely increases the probability of committing Type I error compared to a regular A/B testing without early stopping.*

(Challenging) Q3.4. Extending the Early Stopping Simulation

rubric={accuracy:2,viz:2}

Given your results in **Q3.2** and **Q3.3**, run another four simulations just as in **Q3.3** but varying the significance level α with the following values: `0.001`, `0.005`, `0.01`, `0.02`, `0.04`, and `0.05`. Recall you will be obtaining 1000 A/B tests per simulation **under the early stopping framework**. For each of these simulations, obtain the **proportion** of those 1000 A/B tests where the null hypothesis was rejected.

> **Heads-up:** Note you should assume that both treatment groups have the same population standard deviation of 15 seconds. Increase your group sample by 2 replicates, beginning 2 with a final sample size of 100 elements per group. **These simulations still assume that there is NO difference between both treatments (i.e., H_0 is true in fact for the population).**

Store your results in a data frame called `significance_results` with columns:

- `alpha` (containing `0.001`, `0.005`, `0.01`, `0.02`, `0.04`, and `0.05`), and
- `proportion` (the **proportion** mentioned above per simulation).

```r
# Do not delete this line.
set.seed(554)

significance_results <- NULL

# BEGIN SOLUTION
# Function for computing the proportion of rejected null hypotheses
# per simulation of 1000 replicates
early_stopping_t_test_alpha <- function(n, effect_size, sd, sample_increase,
                                        alpha, reps = 1000) {
  pmap_df(
    list(
      rep(n, reps), rep(effect_size, reps), rep(sd, reps), rep(sample_increase, reps),
      rep(alpha, reps)
    ),
    early_stopping_t_test
  ) %>%
    summarise(
      proportion = mean(significant),
      avg_sample_tr = mean(increasing_size)
    ) %>%
    select(proportion, avg_sample_tr)
}

# Running the four simulations
significance_results <- tibble(
  alpha = c(0.001, 0.005, 0.01, 0.02, 0.04, 0.05),
  proportion = NA,
  avg_sample_tr = NA
)

for (i in 1:nrow(significance_results)) {
  results <- early_stopping_t_test_alpha(
    100, 0, 15, 2,
    significance_results$alpha[i]
  )
  significance_results$proportion[i] <- results$proportion
  significance_results$avg_sample_tr[i] <- results$avg_sample_tr
```

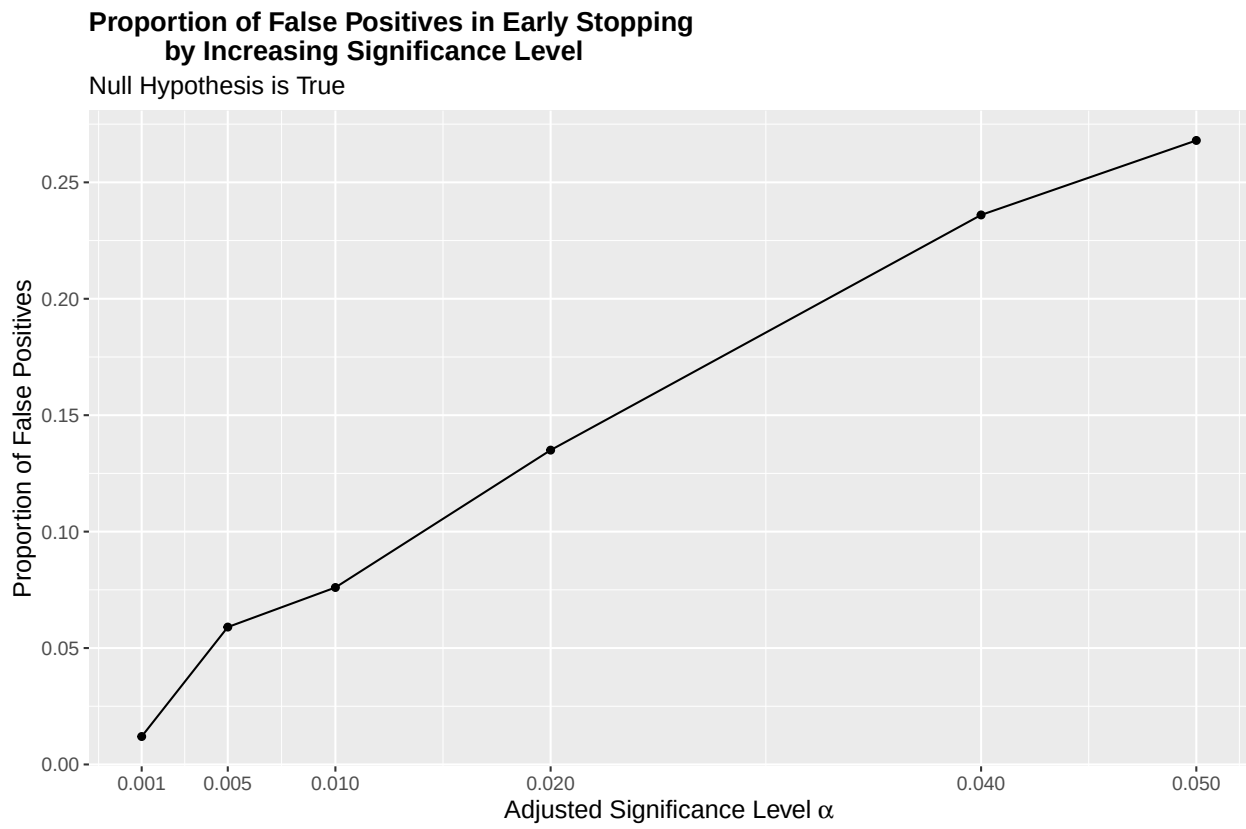
```r
}

significance_results
```

```
## # A tibble: 6 x 3
##   alpha proportion avg_sample_tr
##   <dbl>      <dbl>         <dbl>
## 1 0.001      0.012          99.0
## 2 0.005      0.059          95.6
## 3 0.01       0.076          94.2
## 4 0.02       0.135          89.7
## 5 0.04       0.236          81.8
## 6 0.05       0.268          78.9
```

```r
# END SOLUTION
```

Now, using `significance_results`, plot `alpha` (x-axis) versus `proportion` (y-axis).

```r
# BEGIN SOLUTION
significance_results_plot <- significance_results %>%
  ggplot(aes(x = alpha, y = proportion)) +
  geom_point() +
  geom_line() +
  scale_x_continuous(breaks = c(0.001, 0.005, 0.01, 0.02, 0.04, 0.05)) +
  scale_y_continuous(breaks = seq(0, 0.25, 0.05)) +
  theme(
    plot.title = element_text(size = 13.5, face = "bold"),
    plot.subtitle = element_text(size = 13),
    axis.text.x = element_text(size = 10, angle = 0),
    axis.text.y = element_text(size = 10, angle = 0),
    axis.title = element_text(size = 13)
  ) +
  ggtitle("Proportion of False Positives in Early Stopping
          by Increasing Significance Level",
    subtitle = "Null Hypothesis is True"
  ) +
  labs(
    x = expression("Adjusted Significance Level" ~ alpha),
    y = "Proportion of False Positives"
  )
significance_results_plot
```




```
# END SOLUTION
```

(Challenging) Q3.5. Multiple Simulations with Early Stopping

rubric={accuracy:2,viz:2}

Now, suppose that **the null hypothesis is not true** and **there is a difference in dwell time between both ads**. Under an early stopping framework via a function called `early_stopping_t_test_alpha()` (which you will code up), you will run different simulations under a combination of different settings (as explained in the below code). **Run 1000 replicates per setting combination.**

> **Heads-up:** You will assume that both treatment groups have the same population standard deviation of 15 seconds. Also, we use a raw $\alpha = 0.05$ but adjusted by the planned number of multiple peekings (as seen in column `alpha` in `significance_results_2`).

Note that the results data frame `significance_results_2` also has the column `multiple_peekings` (not to be used by `early_stopping_t_test_alpha()`, but necessary for a post-simulation plotting).

By setting scenario, you need to deliver the following in `significance_results_2`:

- `proportion`: The proportion of those 1000 replicates with statistically significant results (i.e., the power!).
- `avg_sample_tr`: The average early-stopped sample size by treatment.

  Hint: The function `early_stopping_t_test_alpha()` will basically call `early_stopping_t_test()` to be repeated `1000` times (i.e., `1000` replicates per scenario or row in `significance_results_2`) while computing the observed power and the average early-stopped sample size per treatment. You might have coded an analogous function in Q3.4; feel free to use it here as long it provides the necessary outputs.

Running this set of 96 simulations will take around 6 minutes to complete.

```r
# Run this code before proceeding.
# First varying setting, sample size by treatment.
n <- c(100, 200, 400, 800)
# Second varying setting, difference between both treatments in ad dwell times.
effect_size <- c(1, 2.5, 5, 10)
# Third varying setting, sample size increase in early stopping framework.
# Function early_stopping_t_test_alpha() will start checking the testing
# results beginning this sample increase.
sample_increase <- c(2, 4, 5, 10, 20, 50)
# Combining this three settings in a grid.
significance_results_2 <- expand.grid(n, effect_size, sample_increase)
colnames(significance_results_2) <- c(
  "n", "effect_size",
  "sample_increase"
)

significance_results_2 <- significance_results_2 %>%
  mutate(
    multiple_peekings = n / sample_increase,
    alpha = 0.05 / multiple_peekings,
    proportion = NA,
    avg_sample_tr = NA
  )

# BEGIN SOLUTION

# early_stopping_t_test_alpha() was coded in Q3.4

for (i in 1:nrow(significance_results_2)) {
  results <- early_stopping_t_test_alpha(
    n = significance_results_2$n[i],
    effect_size = significance_results_2$effect_size[i],
    sd = 15,
    sample_increase = significance_results_2$sample_increase[i],
    alpha = significance_results_2$alpha[i]
  )
  significance_results_2$proportion[i] <- results$proportion
  significance_results_2$avg_sample_tr[i] <- results$avg_sample_tr
}

# END SOLUTION

head(significance_results_2)
```

```
##     n effect_size sample_increase multiple_peekings    alpha proportion
## 1 100         1.0               2                50 0.001000      0.023
## 2 200         1.0               2               100 0.000500      0.014
## 3 400         1.0               2               200 0.000250      0.025
## 4 800         1.0               2               400 0.000125      0.035
## 5 100         2.5               2                50 0.001000      0.073
## 6 200         2.5               2               100 0.000500      0.118
##   avg_sample_tr
## 1        98.458
## 2       198.460
```

```
## 3       393.660
## 4       784.834
## 5        96.194
## 6       188.936
```

Finally, using `significance_results_2`, plot the results of your different simulations in a grid as follows:

- A column of four plots (one by **planned sample size by treatment**) with `multiple_peekings` on the x-axis and the power of the test on the y-axis.
- A column of four plots (one by **planned sample size by treatment**) with `multiple_peekings` on the x-axis and the early-stopped `avg_sample_tr` on the y-axis.

All plots should have four different coloured lines (one per `effect_size`). Do not forget to include the corresponding legend.

```r
# BEGIN SOLUTION
# Function for plotting power by sample size.
plot_function_power <- function(data, title, multiple_peekings, limit_x) {
  data %>%
    ggplot(aes(
      x = multiple_peekings,
      y = proportion,
      colour = as.factor(effect_size)
    )) +
    geom_point(size = 2) +
    geom_line(linewidth = 1) +
    theme(
      plot.title = element_text(size = 27, face = "bold"),
      plot.subtitle = element_text(size = 26),
      axis.text = element_text(size = 20),
      axis.title = element_text(size = 26),
      legend.position = "none"
    ) +
    labs(x = "Number of Peeks in A/B Testing", y = "Power") +
    coord_cartesian(xlim = limit_x, ylim = c(0, 1)) +
    ggtitle(title) +
    scale_colour_brewer(palette = "Dark2") +
    scale_x_continuous(breaks = multiple_peekings)
}

# Function for plotting average early-stopped sample sizes.
plot_function_size <- function(data, multiple_peekings,
                               limit_x, limit_y) {
  data %>%
    ggplot(aes(
      x = multiple_peekings,
      y = avg_sample_tr,
      colour = as.factor(effect_size)
    )) +
    geom_point(size = 2) +
    geom_line(linewidth = 1) +
    theme(
      plot.title = element_text(size = 27, face = "bold"),
      plot.subtitle = element_text(size = 26),
      axis.text = element_text(size = 20),
      axis.title = element_text(size = 26),
```

```r
      legend.position = "bottom",
      legend.text = element_text(
        size = 26,
        margin = margin(r = 1, unit = "cm")),
      legend.title = element_text(size = 26, face = "bold"),
      legend.key.size = unit(2, "cm")
    ) +
    labs(
      x = "Number of Peeks in A/B Testing",
      y = "Average Early-Stopped n"
    ) +
    coord_cartesian(xlim = limit_x, ylim = limit_y) +
    scale_colour_brewer(palette = "Dark2") +
    scale_x_continuous(breaks = multiple_peekings) +
    labs(colour = "Dwell Time Increase\n with New Ad") +
    scale_colour_brewer(
      palette = "Dark2",
      labels = c("+1 s", "+2.5 s", "+5 s", "+10 s")
    )
}

early_power_plot_n_100 <- plot_function_power(
  data = significance_results_2 %>%
    filter(n %in% 100),
  title = "n = 100 per Treatment",
  multiple_peekings = c(2, 5, 10, 20, 25, 50),
  limit_x = c(2, 50)
)

early_size_plot_n_100 <- plot_function_size(
  data = significance_results_2 %>%
    filter(n %in% 100),
  multiple_peekings = c(2, 5, 10, 20, 25, 50),
  limit_x = c(2, 50),
  limit_y = c(0, 100)
)
```

```
## Scale for colour is already present.
## Adding another scale for colour, which will replace the existing scale.
```

```r
early_power_plot_n_200 <- plot_function_power(
  data = significance_results_2 %>%
    filter(n %in% 200),
  title = "n = 200 per Treatment",
  multiple_peekings = c(4, 10, 20, 40, 50, 100),
  limit_x = c(4, 100)
)

early_size_plot_n_200 <- plot_function_size(
  data = significance_results_2 %>%
    filter(n %in% 200),
  multiple_peekings = c(4, 10, 20, 40, 50, 100),
  limit_x = c(4, 100),
  limit_y = c(0, 200)
```

```
)
```

```
## Scale for colour is already present.
## Adding another scale for colour, which will replace the existing scale.
```

```
early_power_plot_n_400 <- plot_function_power(
  data = significance_results_2 %>%
    filter(n %in% 400),
  title = "n = 400 per Treatment",
  multiple_peekings = c(8, 20, 40, 80, 100, 200),
  limit_x = c(8, 200)
)

early_size_plot_n_400 <- plot_function_size(
  data = significance_results_2 %>%
    filter(n %in% 400),
  multiple_peekings = c(8, 20, 40, 80, 100, 200),
  limit_x = c(8, 200),
  limit_y = c(0, 400)
)
```

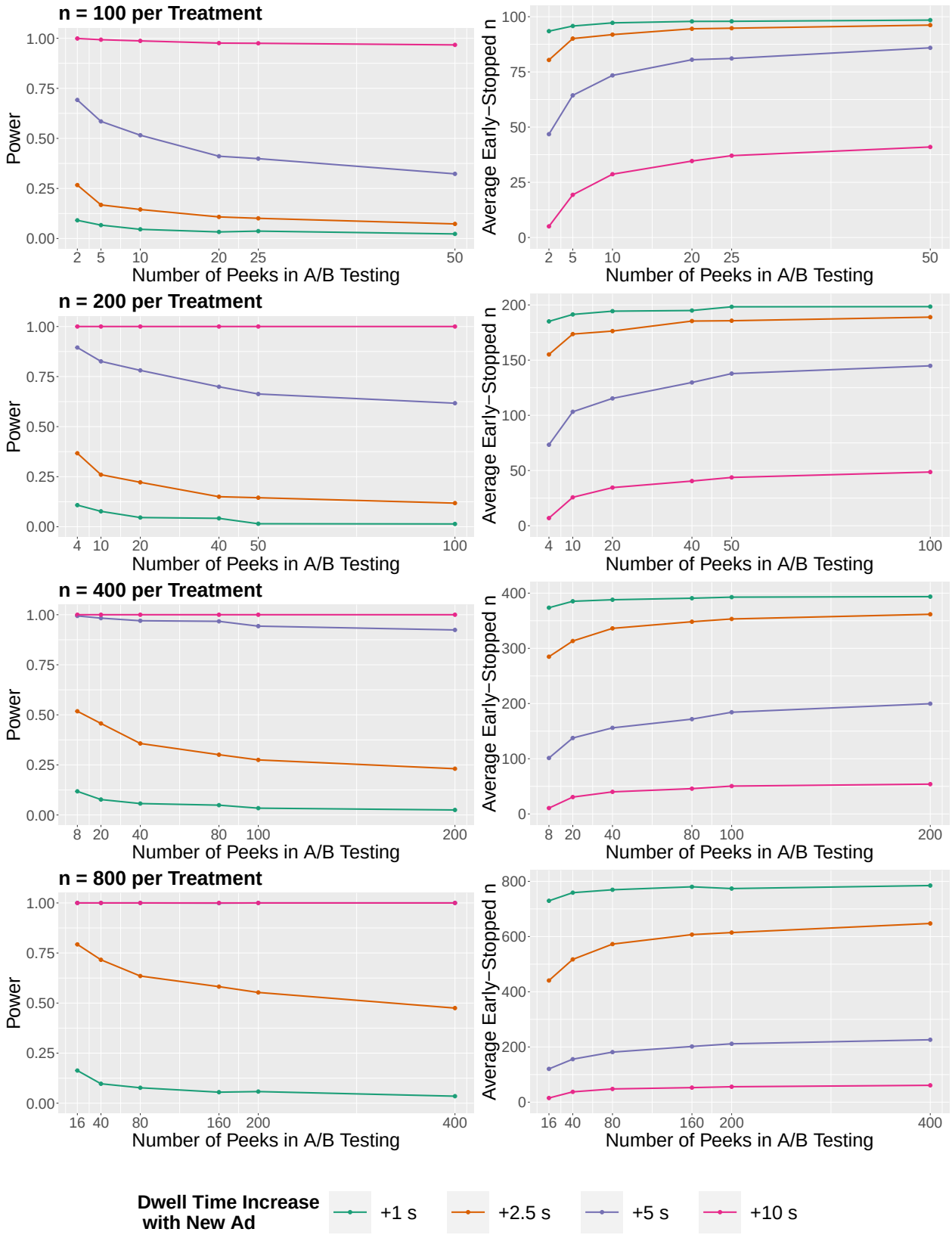
```
## Scale for colour is already present.
## Adding another scale for colour, which will replace the existing scale.
```

```
early_power_plot_n_800 <- plot_function_power(
  data = significance_results_2 %>%
    filter(n %in% 800),
  title = "n = 800 per Treatment",
  multiple_peekings = c(16, 40, 80, 160, 200, 400),
  limit_x = c(16, 400)
)

early_size_plot_n_800 <- plot_function_size(
  data = significance_results_2 %>%
    filter(n %in% 800),
  multiple_peekings = c(16, 40, 80, 160, 200, 400),
  limit_x = c(16, 400),
  limit_y = c(0, 800)
)
```

```
## Scale for colour is already present.
## Adding another scale for colour, which will replace the existing scale.
```

```
legend <- get_legend(early_size_plot_n_100)

plots <- plot_grid(early_power_plot_n_100,
  early_size_plot_n_100 +
    theme(legend.position = "none"),
  early_power_plot_n_200,
  early_size_plot_n_200 +
    theme(legend.position = "none"),
  early_power_plot_n_400,
  early_size_plot_n_400 +
    theme(legend.position = "none"),
  early_power_plot_n_800,
  early_size_plot_n_800 +
```

```r
    theme(legend.position = "none"),
  ncol = 2
)

plot_grid(plots, legend, nrow = 2, rel_heights = c(1, .1))
```




```
# END SOLUTION
```

(Challenging) Q3.6. Comment on Your Findings

rubric={reasoning:4}

In no more than four paragraphs, based on **all** your simulation findings in this **Exercise 3**, explain the problems associated with implementing early stopping in an experiment. Moreover, explain a possible course of action to **correct** this.

ANSWER:

When we run a statistical test, we have to choose a significance level, α. This significance level corresponds to the probability of rejecting the null hypothesis when the null hypothesis is true (we call this a Type I error). Thus if we set $\alpha = 0.05$, we are essentially saying that we accept that there is a 5% chance that we will falsely reject the null even if the null hypothesis is true.

Suppose we run an experiment and monitor the p-value after every or even many observations when the null hypothesis is true. In that case, we inflate our Type I error beyond α each time we do this. The more times we do this, the worse it gets. In doing this, we may end up stopping the experiment early because of a false positive result.

Now, suppose we run an experiment and monitor the p-value after every or even many observations when the null hypothesis is not true. We might be tempted to say this will not affect the power of our test. Nevertheless, it could affect it, especially for scenarios where the effect size (i.e., the difference between both treatments) is small.

This is akin to creating a multiple-testing problem with our analysis. Given this, we can use certain methods to address these problems, for example, a Bonferroni correction. However, when we do this, we can lose power in our study as the Bonferroni correction can be quite conservative. Hence, we need to optimize the number of times we peek to apply a Bonferroni correction without being overly conservative. Therefore, we will not lose the power to detect a true effect (if there is one).

Exercise 4: Introduction to Observational Studies in Causal Inference

Q4.1. Confounders in an Observational Study

rubric={reasoning:6}

A group of past MDS students wanted to know whether a person's self-rated enjoyment of the MDS program (X) had any causal influence on their expected salary upon graduation (Y).

They planned to run a survey to answer this question, which would include (**at least**) the following two questions:

1. What is your salary expectation after graduation in CAD?
   - Less than $60,000
   - $60,000 to $80,000
   - $80,001 to $100,000
   - $100,001 to $120,000
   - More than $120,000
2. What is your self rated enjoyment of MDS on a scale from 1 to 4 (4 being very happy with MDS and 1 being not happy at all with MDS)?
   - 1
   - 2
   - 3
   - 4

Given that this question can only be realistically answered with an observational study, confounders should be considered when collecting the data (via the planned survey) and carrying out the analysis.

What confounders do you think should be considered?

> **Note:** You do have domain expertise here to answer this. When answering write one or two sentences to justify the inclusion of each confounder. Include, **at least**, **four confounders** in your answer.

ANSWER:

Terms	Definitions
Salary before MDS	Previous salary could be confounding as it will affect future salary expectations and affect happiness in the program. For example, a low prior salary may make a person happier to enter MDS and have a potentially higher future salary.
Years of professional work experience	Years of work experience could be confounding as it will affect future salary expectations. A person with more years of work experience might not be as happy to enter MDS.
Confidence in data science skills before starting MDS	Confidence in data science skills before starting MDS would likely affect future salary expectations and directly influence how happy they were in the program. If a person is more confident, then they will likely have an easier time and be happier.
Frequency of completing optional questions in labs	Doing optional questions in labs indicates a person is likely finishing all of the required content with relative ease or likes a challenge. If this is the case, they could be happier with MDS and expect a larger salary to reflect their skill set.
Actively applying for jobs in block 6 of MDS	If people are actively applying for jobs in block 6 of MDS, it could be affirming that MDS has prepared them adequately or the opposite. This would affect their feelings about the program and, consequently, salary expectations.
Current happiness level	A baseline for happiness. If they are not happy, they are not happy with MDS, and then maybe they do not expect much of a salary.
Frequency of attending MDS career events	Attending career events could increase confidence in job opportunities and, subsequently, salary. If the events are useful, then the feelings about MDS would likely change for the better.

Q4.2. Causal Analysis in an Observational Study

rubric={reasoning:6}

Given the question being asked in **Q4.1** and the confounders you answered, how would you analyze the data once you have collected it?

Answer in two or three sentences.

ANSWER:

Possible Solution:

> *Since the survey will be designed to include questions that identify potential confounding variables, I would plan to account for these confounding variables in our statistic analysis via stratification. I would form subgroups of survey responses with the same values of the confounding variables, with particular consideration to these groups' sizes, to assess whether there are sufficient observations in each aggregated group.*
>
> *Within each stratum, I would assess the association between expected annual salary after graduation and the self-reported enjoyment of the MDS program. I could do this using an ordinal logistic regression where the response is expected salary, and the explanatory variable is the self-reported enjoyment of MDS. Ordinal logistic regression would be chosen due to the ordered but unevenly spaced binning of the response variable.*
>
> *I would interpret the linear models' results across the strata and form a conclusion on the significance and size of the effect of MDS enjoyment on expected salary, having controlled for the confounders.*

Submission

Congratulations! You are done the lab. Do not forget to:

- Knit the assignment to generate the `.pdf` file and push everything to your Github repo.
- Double check all the figures, texts, equations are rendered properly in the `.pdf` file
- **Submit the `.pdf` file to Gradescope.**

Attribution

- Exercises 1 and 2 in this lab were inspired and derived from OpenIntro Statistics by David Diez, Mine Cetinkaya-Rundel, and Christopher Barr.
- Exercise 3 is based on the work of **Early stopping** by Tim Head.